

B. Can you Traverse-2?

time limit per test: 1 second

memory limit per test: 1024 megabytes

You are given an **undirected unweighted** graph with N cities and M roads. The cities are numbered from 1 to N . The graph is connected, and contains no self-loops or multiple edges.

Your task is to perform a Depth-First Search (DFS) starting from node 1 and print the order in which the nodes are visited.

Pseudocode of DFS

The depth-first-search procedure below assumes that the input graph $G = (V, E)$ is represented using adjacency lists.

```
colourInitializing(G):
    for each vertex u in G.V:
        u.colour = 0
```

```
DFS(G,u):
    u.colour = 1
    for each v in G.Adj[u]:
        if v.colour == 0:
            DFS(G,v)
```

Important Notes for Java Language

- **General Note:** Printing inside a recursive DFS is logically valid. However, expressions such as `pw.write(node + " ")` create temporary strings and perform a separate output operation for every visited node, adding unnecessary overhead. This overhead may be handled differently across Java versions. Therefore, code accepted in Java 8 may fail in Java 21 when the recursion depth is already very large.

• Recommended Approach

Store or append the traversal result during DFS, then print the complete result after the traversal finishes instead of printing inside every recursive call.

Important Notes for Python Language

- **General Note:** The following instructions apply to all problems that use deep recursion in Python, not only this problem. The recursion limit must be adjusted according to the maximum possible recursion depth of each problem.

Python has a default recursion limit (usually around 1000) to prevent stack overflow from runaway recursion. As a result, if you are using recursion to solve a problem, you must increase the recursion limit manually.

• How to Increase the Recursion Depth

```
import sys
```

CSE221 Assignment 05 Summer 2026

Contest is running

4 weeks

Contestant



→ Submit?

 Language: Java 21 64bit

 Choose file: Choose File No file chosen


Success!



Submit

```
sys.setrecursionlimit(#set_the_value)
```

• What Value to Set

Setting the recursion depth too high can cause a stack overflow or crash your program. This happens because each recursive call uses stack memory, and the system allocates only a limited amount of memory for the stack. If the recursion goes deeper than the system can handle, it may lead to a segmentation fault. Check what the maximum recursion depth might be in your problem. In this problem, the maximum number of nodes the input graph can have is 2×10^5 , so we set the limit accordingly. We add a small buffer (+5, or sometimes +10, +50, etc.) to avoid edge-case runtime errors.

```
import sys
sys.setrecursionlimit(2*100000+5)
```

• Choose Python over PyPy to Avoid MLE/RTE

While PyPy is generally faster due to Just-In-Time (JIT) compilation, it also uses more memory. In recursion-heavy problems, PyPy may consume significantly more stack memory per call compared to CPython. This can lead to Memory Limit Exceeded (MLE) errors on some platforms, especially when recursion depth is high. Therefore, for problems that involve deep recursion and strict memory constraints, it is safer to submit your solution using Python (CPython) rather than PyPy.

Input

The first line contains two integers N and M ($1 \leq N \leq 2 \times 10^5, 1 \leq M \leq 3 \times 10^5$) — the number of cities and the total number of roads.

The second line contains M integers $u_1, u_2, u_3 \dots u_m$ ($1 \leq u_i \leq N$) — where the i -th integer represents the node that is one endpoint of the i -th edge.

The third line contains M integers $v_1, v_2, v_3 \dots v_m$ ($1 \leq v_i \leq N$) — where the i -th integer represents the node that is other endpoint of the i -th edge.

The i -th edge of this graph is between the i -th node in the second line and the i -th node in the third line.

Output

Print the DFS traversal starting from node 1 as a space-separated list of visited nodes. If there are multiple DFS path traversal order, you may print any.

Examples

input	Copy
4 3 1 3 1 4 2 3	
output	Copy
1 3 2 4	
input	Copy
6 8 1 5 3 4 6 1 6 4 5 3 4 6 2 3 3 1	
output	Copy
1 3 4 6 2 5	
input	Copy
5 7 5 1 3 2 4 4 4 1 3 2 4 1 3 5	

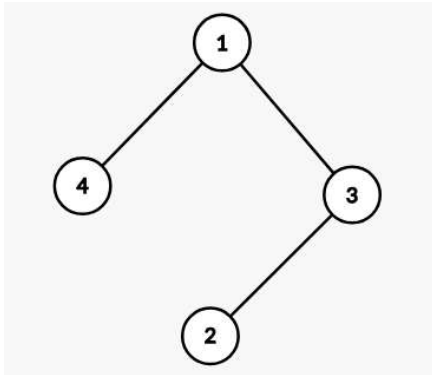
output

Copy

1 3 2 4 5

Note

In the first Sample Input, the graph looks like this:



There are two valid paths that follow the DFS order: 1 4 3 2 and 1 3 2 4. You may print either one of these paths in the output.

[Codeforces](#) (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform
Server time: Jul/19/2026 14:43:24^{UTC+6} (i1).
Desktop version, switch to [mobile version](#).
[Privacy Policy](#) | [Terms and Conditions](#)

Supported by

